



UNIVERSITE HASSAN II DE CASABLANCA
FACULTE DES SCIENCES BEN M'SICK

Documentation PFE - PDF 02/10

Frontend Angular Complet

Angular 17 + TypeScript + Composants + Routing + Services

Realise par :

AKRAM BELMOUSSA - Backend & Integration NLP

ZAKARIA - Frontend Angular & UX/UI

NOUHAILA - Base de donnees & Contenu FAQ

MAY 2026

Sommaire

Chapitre 1 - C'est quoi Angular ?	3
Chapitre 2 - TypeScript : les bases	7
Chapitre 3 - Anatomie d'un composant Angular	11
Chapitre 4 - Templates et data binding	16
Chapitre 5 - Services et injection de dependance	20
Chapitre 6 - HttpClient et appels API	24
Chapitre 7 - Routing et lazy loading	27
Chapitre 8 - Signals (Angular 17)	31
Chapitre 9 - Layout shell explique	34
Chapitre 10 - Les 9 pages decortiquees	37
Chapitre 11 - Mode sombre et theme service	44
Chapitre 12 - Animations et responsive	47
Chapitre 13 - Build et deploiement	50

Chapitre 1 - C'est quoi Angular ?

1.1 Definition courte

Angular est un framework JavaScript (plus precisement TypeScript) developpe par **Google** depuis 2010. Il sert a construire des SPA (Single Page Applications) de facon structuree.

1.2 Histoire en bref

Annee	Version	Note
2010	AngularJS (v1)	Premiere version, JavaScript pur
2016	Angular 2	Reecriture complete en TypeScript
2018	Angular 6+	Stabilisation, releases tous les 6 mois
2023	Angular 17	Standalone components officialise
2024	Angular 18	Zoneless preview, ameliorations

■ Notre projet utilise **Angular 17.3**, sorti en mars 2024. C'est une version stable, supportee, avec toutes les nouveautes modernes (standalone, signals, control flow).

1.3 Pourquoi Angular ?

Angular est un framework **opinionated** : il impose une structure et une maniere de faire. C'est a la fois sa force et sa faiblesse.

Forces	Faiblesses
Structure imposee = code uniforme	Courbe d'apprentissage raide
Routing, forms, HTTP integres	Plus verbeux que React
TypeScript par default (typage fort)	Bundle initial plus gros
CLI puissant (ng generate)	Moins flexible que Vue/React
Soutenu par Google	Conventions parfois rigides
Excellent pour grosses apps	Overkill pour petits sites

■ **ANALOGIE** : Angular est comme un **uniforme scolaire** : tout le monde est habille pareil, c'est moins fun mais on s'y retrouve. React est comme des **vetements libres** : plus de liberte, mais chacun fait son style et c'est moins coherent en equipe.

1.4 Que produit Angular ?

Quand on lance `npm run build`, Angular produit :

- Un fichier **index.html** minimal (le 'P' de SPA)
- Plusieurs fichiers **.js** compresses (le code de l'application)

- Un fichier **styles.css** minifie
- Des **chunks** pour les pages lazy-loaded
- Des assets (images, fonts, logos)

Ces fichiers sont servis par un serveur web (en dev: Angular dev server sur port 4200, en prod: nginx/Apache). Le navigateur les telecharge, execute le JS, et tout s'anime.

Chapitre 2 - TypeScript : les bases

2.1 C'est quoi TypeScript ?

TypeScript est un **sur-ensemble** de JavaScript developpe par Microsoft. Il ajoute un systeme de types statiques a JavaScript.

2.2 JavaScript vs TypeScript

```
// JavaScript (typage dynamique)
function ajouter(a, b) {
  return a + b;
}
ajouter(2, 3);           // 5 - OK
ajouter("2", "3");      // "23" - bug silencieux !

// TypeScript (typage statique)
function ajouter(a: number, b: number): number {
  return a + b;
}
ajouter(2, 3);           // 5 - OK
ajouter("2", "3");      // ERREUR DE COMPILATION !
```

2.3 Les types de base

Type	Exemple	Description
string	"Hello"	Chaine de caracteres
number	42, 3.14	Nombre (entier ou decimal)
boolean	true, false	Vrai ou faux
null	null	Valeur absente intentionnellement
undefined	undefined	Non defini
any	(tout)	Echappe le typage (a eviter !)
void	(rien)	Pour les fonctions sans retour
number[]	[1, 2, 3]	Tableau de nombres
string[]	["a", "b"]	Tableau de strings
{name: string}	{name: "Akram"}	Objet typé

2.4 Interfaces

Les interfaces definissent la forme d'un objet :

```
interface Message {
  id: number;
  sender: 'bot' | 'user'; // type union
  text: string;
  timestamp: Date;
```

```
intent?: string;           // ? = optionnel
confidence?: number;
}

// Utilisation
const msg: Message = {
  id: 1,
  sender: 'user',
  text: 'Bonjour',
  timestamp: new Date()
  // intent et confidence sont optionnels
};
```

2.5 Classes

```
class ChatService {
  // Proprietes typees
  private apiUrl: string = '/api';

  // Constructeur avec parametres typés
  constructor(private http: HttpClient) {}

  // Methode typée
  sendMessage(message: string): Observable<ChatResponse> {
    return this.http.post<ChatResponse>(
      `${this.apiUrl}/chat`,
      { message }
    );
  }
}
```

2.6 Les avantages du typage

- + **Erreurs detectees a la compilation** au lieu d'au runtime
- + **Autocompletion** dans l'IDE (VSCode propose les methodes/proprietes)
- + **Refactoring** sur : on renomme une variable, le compilateur trouve tous les usages
- + **Documentation vivante** : le type EST la documentation
- + **Confiance** : si ca compile, ca a une bonne chance de marcher

Chapitre 3 - Anatomie d'un composant Angular

3.1 Qu'est-ce qu'un composant ?

Un **composant** est l'unité de base d'une application Angular. Il combine 3 choses :

- **Template HTML** : ce qu'on voit à l'écran
- **Logique TypeScript** : le comportement
- **Styles CSS** : l'apparence

■ **ANALOGIE** : Un composant est comme une **brique LEGO**. Tu peux combiner plusieurs briques pour construire des structures complexes. Notre app a 28 briques (composants) au total.

3.2 Exemple complet - typing-indicator

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-typing-indicator', // balise HTML personnalisée
  standalone: true,                // composant autonome
  template: `
    <div class="typing-indicator">
      <div class="typing-dot"></div>
      <div class="typing-dot"></div>
      <div class="typing-dot"></div>
    </div>
  `
})
export class TypingIndicatorComponent {}
```

Maintenant, dans un autre composant, on peut l'utiliser comme une balise HTML :

```
<app-typing-indicator></app-typing-indicator>
// ou simplement
<app-typing-indicator />
```

3.3 Le decorateur @Component

@Component est un **decorateur**. C'est une syntaxe TypeScript qui permet d'attacher des métadonnées à une classe. Angular les utilise pour comprendre ce qu'est cette classe.

Propriété	Rôle
selector	Nom de la balise HTML (ex:)
standalone	true = pas besoin de NgModule (Angular 17+)
imports	Liste des dépendances (autres composants, modules)
template	HTML inline

templateUrl	Ou fichier HTML externe
styles	CSS inline
styleUrl	Ou fichier CSS externe

3.4 @Input et @Output

Pour communiquer entre composants :

@Input : recevoir des donnees du parent

```
// Enfant : message-bubble.component.ts
export class MessageBubbleComponent {
  @Input() message!: Message; // ! = sera defini
}

// Parent : chat-window.component.html
<app-message-bubble [message]="msg" />
```

@Output : envoyer des evenements au parent

```
// Enfant : input-bar.component.ts
export class InputBarComponent {
  @Output() sendMessage = new EventEmitter<string>();

  onClick() {
    this.sendMessage.emit(this.message);
  }
}

// Parent : chat-window.component.html
<app-input-bar (sendMessage)="handleSend($event)" />
```

3.5 Les hooks de cycle de vie

Angular appelle des methodes a des moments precis du cycle de vie d'un composant :

Hook	Quand est-il appele
ngOnInit	Apres l'initialisation (1 fois)
ngOnChanges	Quand un @Input change
ngAfterViewInit	Apres le rendu du template
ngAfterViewChecked	Apres chaque check du template
ngOnDestroy	Juste avant la destruction du composant

```
export class ChatWindowComponent implements OnInit, AfterViewChecked {
  ngOnInit() {
    // Initialise le 1er message d'accueil
    this.messages.push({ id: 0, sender: 'bot', text: 'Bonjour !' });
  }

  ngAfterViewChecked() {
    // Scroll auto vers le bas a chaque nouveau message
    if (this.messagesEnd) {
```

```
        this.messagesEnd.nativeElement.scrollIntoView();  
    }  
}  
}
```

Chapitre 4 - Templates et data binding

4.1 Interpolation : {{ }}

Pour afficher une valeur de la classe dans le template :

```
// TS
export class Component {
  name = 'Akram';
  count = 42;
}

// HTML
<p>Bonjour {{ name }}, j'ai {{ count }} messages.</p>
// Affiche : Bonjour Akram, j'ai 42 messages.
```

4.2 Property binding : [propriete]

Pour lier une propriete HTML a une valeur dynamique :

```
<img [src]="logoUrl" [alt]="logoAlt" />
<button [disabled]="isLoading">Envoyer</button>
<div [class.active]="isSelected">..</div>
<div [style.color]="textColor">..</div>
```

4.3 Event binding : (evenement)

Pour reagir aux evenements utilisateur :

```
<button (click)="handleClick()">Cliquer</button>
<input (input)="handleInput($event)" />
<input (keydown.enter)="handleEnter()" />
<form (submit)="handleSubmit()">..</form>
```

4.4 Two-way binding : [(ngModel)]

Pour synchroniser une variable avec un champ formulaire :

```
// TS
export class Component {
  username = '';
}

// HTML
<input [(ngModel)]="username" placeholder="Nom..." />
<p>Bonjour {{ username }}</p>
// Quand on tape dans l'input, username se met a jour
// ET le paragraphe s'update en temps reel
```

4.5 Directives structurales

@if / @for / @switch (Angular 17+)

```
@if (user) {  
  <p>Connecte : {{ user.name }}</p>  
} @else {  
  <p>Non connecte</p>  
}  
  
@for (item of items; track item.id) {  
  <li>{{ item.name }}</li>  
}  
  
@switch (status) {  
  @case ('online') { <span>En ligne</span> }  
  @case ('away')   { <span>Absent</span> }  
  @default         { <span>Hors ligne</span> }  
}
```

Anciennes directives *ngIf / *ngFor

```
<p *ngIf="user">Connecte : {{ user.name }}</p>  
<p *ngIf="!user">Non connecte</p>  
  
<li *ngFor="let item of items">{{ item.name }}</li>
```

■ Notre projet utilise principalement les **anciennes directives** (*ngIf, *ngFor). C'est OK et toujours supporté. Pour un nouveau projet, prefere @if / @for.

Chapitre 5 - Services et injection de dependance

5.1 C'est quoi un service ?

Un **service** est une classe qui contient de la logique reusable. Typiquement : appels HTTP, gestion d'etat global, calculs metier.

■ **ANALOGIE** : Si les composants sont des **vendeurs** dans un magasin, les services sont les **fournisseurs**. Les vendeurs demandent aux fournisseurs ce dont ils ont besoin. Tous les vendeurs partagent les memes fournisseurs.

5.2 Exemple : ChatService

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

@Injectable({ providedIn: 'root' }) // singleton dans toute l'app
export class ChatService {
  private apiUrl = '/api';

  constructor(private http: HttpClient) {}

  sendMessage(message: string, sessionId: string): Observable<ChatResponse> {
    return this.http.post<ChatResponse>(
      `${this.apiUrl}/chat`,
      { message, session_id: sessionId }
    );
  }

  sendFeedback(payload: FeedbackPayload) {
    return this.http.post(`${this.apiUrl}/feedback`, payload);
  }
}
```

5.3 Injection de dependance

L'injection de dependance (DI) est le pattern par lequel Angular fournit automatiquement les services aux composants qui en ont besoin.

Methode 1 : Constructor injection (ancien)

```
export class ChatPageComponent {
  constructor(private chatService: ChatService) {}

  send(text: string) {
    this.chatService.sendMessage(text, 'sess').subscribe(...);
  }
}
```

Methode 2 : inject() function (Angular 14+)

```
import { inject } from '@angular/core';

export class ChatPageComponent {
  private chatService = inject(ChatService);

  send(text: string) {
    this.chatService.sendMessage(text, 'sess').subscribe(...);
  }
}
```

[★] Avantage : pas besoin de constructeur, plus concis, fonctionne aussi dans les guards et resolvers (là où il n'y a pas de constructeur).

5.4 providedIn

providedIn: 'root' indique que le service est un **singleton global** : une seule instance partagée dans toute l'application. C'est ce qu'on veut pour 99% des services.

Chapitre 6 - HttpClient et appels API

6.1 Configuration

HttpClient doit etre enregistre au bootstrap de l'app :

```
// main.ts
import { provideHttpClient } from '@angular/common/http';
import { bootstrapApplication } from '@angular/platform-browser';

bootstrapApplication(AppComponent, {
  providers: [
    provideHttpClient(),
    provideAnimations(),
    provideRouter(routes),
  ]
}).catch(err => console.error(err));
```

6.2 Les 4 methodes principales

```
// GET
this.http.get<Filiere[]>('/api/academic/filieres')
  .subscribe(filieres => console.log(filieres));

// POST avec body
this.http.post<ChatResponse>(
  '/api/chat',
  { message: 'Bonjour' }
).subscribe(resp => console.log(resp));

// PUT
this.http.put(`/api/users/${id}`, payload).subscribe();

// DELETE
this.http.delete(`/api/messages/${id}`).subscribe();
```

6.3 Observables et subscribe

HttpClient retourne des **Observables** (RxJS). C'est comme une Promise, mais en plus puissant. Pour declencher la requete, il faut .subscribe().

```
this.chatService.sendMessage('Bonjour', 'sess')
  .subscribe({
    next: (response) => {
      // Succes : la reponse est arrivee
      this.messages.push({ text: response.response, ... });
    },
    error: (err) => {
      // Erreur : reseau, 4xx, 5xx
      console.error('Erreur', err);
    },
    complete: () => {
      // Fini (souvent inutile pour HTTP)
    }
  });
```

6.4 Operateurs RxJS utiles

```
import { catchError, map, of } from 'rxjs';

this.http.get<Filiere[]>('/api/academic/filieres').pipe(
  map(filieres => filieres.filter(f => f.is_active)),
  catchError(err => {
    console.error(err);
    return of([]); // fallback : liste vide
  })
).subscribe(active => console.log(active));
```

Chapitre 7 - Routing et lazy loading

7.1 Configuration des routes

Notre fichier app.routes.ts definit toutes les pages :

```
import { Routes } from '@angular/router';

export const routes: Routes = [
  {
    path: '',
    pathMatch: 'full',
    loadChildren: () =>
      import('./features/dashboard/dashboard.component')
        .then(m => m.DashboardComponent),
    title: 'Accueil - Chatbot FSBM',
  },
  {
    path: 'chat',
    loadChildren: () =>
      import('./features/chat/chat-page.component')
        .then(m => m.ChatPageComponent),
  },
  // ... 7 autres routes
  { path: '**', redirectTo: '' }, // fallback 404
];
```

7.2 Lazy loading explique

loadComponent est la magie : la page n'est **pas chargée au départ**. Quand l'utilisateur clique sur le lien /chat, Angular telecharge LE chunk JS de chat (~20 KB), puis l'affiche.

■ **ANALOGIE** : Au lieu de telecharger un dictionnaire complet (lourd), tu telecharges les pages au fur et a mesure que tu as besoin de chaque mot. Plus rapide au demarrage, et tu ne telecharge que ce que tu utilises vraiment.

7.3 Navigation programmatique

```
import { Router } from '@angular/router';
import { inject } from '@angular/core';

export class HomeComponent {
  private router = inject(Router);

  startChat() {
    this.router.navigate(['/chat']);
  }

  viewFiliere(code: string) {
    this.router.navigate(['/filieres', code]);
  }
}
```

7.4 Liens dans le template

```
<!-- routerLink declenche la navigation -->
<a routerLink="/chat">Ouvrir le chat</a>

<!-- Avec parametres -->
<a [routerLink]="['/filieres', filiere.code]">
  Voir {{ filiere.name }}
</a>

<!-- routerLinkActive ajoute une classe quand actif -->
<a routerLink="/filieres" routerLinkActive="active">
  Filieres
</a>
```

7.5 Recuperer les parametres d'URL

```
import { ActivatedRoute } from '@angular/router';

export class FiliereDetailComponent implements OnInit {
  private route = inject(ActivatedRoute);
  filiere: Filiere | null = null;

  ngOnInit() {
    this.route.params.subscribe(params => {
      const code = params['code']; // recupere :code de l'URL
      this.loadFiliere(code);
    });
  }
}
```

Chapitre 8 - Signals (Angular 17)

8.1 C'est quoi un Signal ?

Un **Signal** est une **valeur reactive** introduite dans Angular 17. Quand sa valeur change, tous les composants qui l'utilisent sont automatiquement mis a jour.

8.2 Creer et utiliser un signal

```
import { signal, computed } from '@angular/core';

export class ThemeService {
  // Creation d'un signal
  readonly theme = signal<'light' | 'dark'>('light');

  // Signal calculé
  readonly isDark = computed(() => this.theme() === 'dark');

  // Lecture (appel comme une fonction)
  current() {
    return this.theme();
  }

  // Modification
  toggle() {
    this.theme.set(this.theme() === 'light' ? 'dark' : 'light');
    // ou
    this.theme.update(v => v === 'light' ? 'dark' : 'light');
  }
}
```

8.3 Dans un template

```
<!-- Lire un signal : appeler comme une fonction -->
<p>Theme actuel : {{ theme.theme() }}</p>

<!-- Conditionnel base sur signal -->
@if (theme.isDark()) {
  <span>■ Mode sombre</span>
} @else {
  <span>■ Mode clair</span>
}

<!-- Bouton qui change le signal -->
<button (click)="theme.toggle()">Changer</button>
```

8.4 Signals vs Observables

Critere	Signal	Observable (RxJS)
Synchrone	OUI (lecture instantanee)	NON (besoin subscribe)
Memoire	Reactive automatique	Manuel

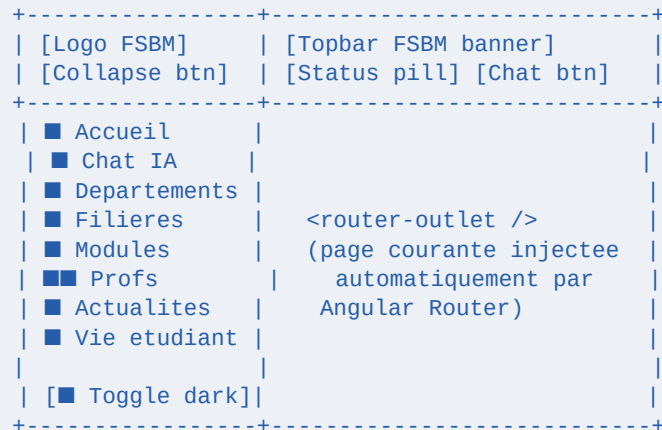
Complexite	Simple	Plus puissant
Use case	Etat local	Streams asynchrones
Cas FSBM	Theme, toggle UI	Appels HTTP

[★] Notre projet utilise les Signals pour le theme service, le toggle LLM, et l'etat local des composants chat. Les Observables restent pour les appels HTTP.

Chapitre 9 - Layout shell explique

9.1 Structure

Le AppShellComponent est le **squelette** partage par toutes les pages. Il contient la sidebar de navigation, la topbar avec le logo FSBM, et un emplacement `<router-outlet/>` ou Angular injecte la page courante.



9.2 Code TypeScript principal

```

@Component({
  selector: 'app-shell',
  standalone: true,
  imports: [CommonModule, RouterLink, RouterLinkActive, RouterOutlet],
  templateUrl: './app-shell.component.html',
  styleUrls: ['./app-shell.component.css'],
})
export class AppShellComponent {
  readonly theme = inject(ThemeService);
  readonly collapsed = signal(false);
  readonly isDark = computed(() => this.theme.theme() === 'dark');

  readonly navItems = [
    { path: '/', icon: '■', label: 'Accueil', exact: true },
    { path: '/chat', icon: '■', label: 'Assistant IA' },
    { path: '/departements', icon: '■', label: 'Departements' },
    { path: '/filieres', icon: '■', label: 'Filieres' },
    { path: '/modules', icon: '■', label: 'Modules' },
    { path: '/professeurs', icon: '■', label: 'Professeurs' },
    { path: '/actualites', icon: '■', label: 'Actualites' },
    { path: '/vie-etudiante', icon: '■', label: 'Vie etudiante' },
  ];

  toggleCollapse() {
    this.collapsed.update(v => !v);
  }
}

```

9.3 Comment ca s'integre

```
// app.component.ts
@Component({
  selector: 'app-root',
  standalone: true,
  imports: [AppShellComponent],
  template: `<app-shell />`, // affiche le shell
})
export class AppComponent {}
```

Chapitre 10 - Les 9 pages decortiquees

Dashboard (/)

Composant : DashboardComponent

Affiche : Hero anime + 8 cartes stats (live academic-service) + 4 dernieres annonces + 3 prochains evenements + 4 actions rapides. Au ngOnInit, lance 3 appels HTTP en parallele (overview, announcements, events). Si backend down, affiche un warning.

Chat (/chat)

Composant : ChatPageComponent + ChatWindowComponent

Interface chat avec messages bubbles, indicator typing, input bar, quick actions. Maintient un sessionId unique. Toggle TF-IDF/LLM en haut a droite. Personnalise via memory backend (genre, nom, historique 10 tours).

Departements (/departements)

Composant : DepartmentsComponent

5 cartes avec couleur par departement (CSS variable --dept-color). Chaque carte affiche chef, email, telephone, lien vers ses filieres.

Filieres (/filieres)

Composant : FilieresComponent

Liste de 25 filieres avec barre de recherche debounee et chips de filtres (Licence/Master/Pro/Recherche). Utilise computed() pour filtrer en temps reel.

Detail filiere (/filieres/:code)

Composant : FiliereDetailComponent

Recupere le code depuis l'URL. Charge filiere par code + modules groupes par semestre. Affiche hero gradient (different selon licence/master), objectifs, debouches, programme S1-S6.

Modules (/modules)

Composant : ModulesComponent

100+ modules avec 3 filtres : recherche texte, filiere (dropdown), semestre (1-6). Carte par module avec code, semestre, ECTS, coefficient, heures cours/TD/TP.

Professeurs (/professeurs)

Composant : ProfessorsComponent

Annuaire de 107 profs paginé (12/page) avec : recherche, filtre departement, filtre grade (PA/PH/PES/Vacataire). Avatar = initiales. Email cliquable.

Actualites (/actualites)

Composant : NewsComponent

Layout 2 colonnes : annonces officielles (colonne large) + evenements a venir (sidebar). Badges colorés selon type (URGENT/EXAMEN/EVENT/VACANCE). Lien inscription evenement si disponible.

Vie etudiante (/vie-etudiante)

Composant : StudentLifeComponent

Grille de 8 clubs avec couleur de bordure selon categorie (SCIENTIFIQUE/TECHNIQUE/CULTUREL/SPORTIF/HUMANITAIRE). Reseaux sociaux cliquables.

Chapitre 11 - Mode sombre et theme service

11.1 Architecture

Le mode sombre repose sur 3 piliers :

- **ThemeService** : signal qui contient 'light' ou 'dark'
- **Variables CSS** dans styles.css : --primary, --bg, --surface, etc.
- **Attribut HTML** : data-theme sur la racine, ecoute par CSS

11.2 Code complet du service

```
import { Injectable, signal } from '@angular/core';

@Injectable({ providedIn: 'root' })
export class ThemeService {
  private readonly STORAGE_KEY = 'fsbm-theme';
  readonly theme = signal<'light' | 'dark'>('light');

  constructor() {
    // 1. Recuperer la preference depuis localStorage
    const stored = localStorage.getItem(this.STORAGE_KEY);
    // 2. Fallback : detecter le theme systeme du navigateur
    const initial = stored || (
      window.matchMedia('(prefers-color-scheme: dark)').matches
        ? 'dark' : 'light'
    );
    this.applyTheme(initial as 'light' | 'dark');
  }

  toggle() {
    this.applyTheme(this.theme() === 'light' ? 'dark' : 'light');
  }

  private applyTheme(value: 'light' | 'dark') {
    this.theme.set(value);
    document.documentElement.setAttribute('data-theme', value);
    localStorage.setItem(this.STORAGE_KEY, value);
  }
}
```

11.3 Variables CSS

```
:root,
[data-theme="light"] {
  --primary:    #1C3F6E;
  --accent:     #16B5A6;
  --bg:         #F4F7FA;
  --surface:    #FFFFFF;
  --text-dark:  #1A2638;
  --border:     #DDE4ED;
```

```
/* ... */  
}  
  
[data-theme="dark"] {  
  --primary:    #5B8FE3;  
  --accent:     #25D0BF;  
  --bg:         #0F1620;  
  --surface:    #1A2333;  
  --text-dark:  #F1F5F9;  
  --border:     #2A3548;  
  /* ... */  
}  
  
/* Tous les composants utilisent les variables */  
.card {  
  background: var(--surface);  
  color: var(--text-dark);  
  border: 1px solid var(--border);  
  transition: background 0.3s, color 0.3s;  
}
```

[★] Quand le service change data-theme sur <html>, TOUTES les variables CSS basculent automatiquement. Les composants n'ont rien à faire - ils utilisent juste var(--primary) etc.

Chapitre 12 - Animations et responsive

12.1 Animations CSS

Notre projet utilise des animations CSS pures (pas Angular Animations API pour rester simple).
Exemple de l'apparition des cartes :

```
@keyframes fadeInUp {
  from { opacity: 0; transform: translateY(12px); }
  to   { opacity: 1; transform: translateY(0); }
}

.fade-in-up {
  animation: fadeInUp 0.4s ease both;
}

/* Delay pour stagger effect */
.stat-card:nth-child(1) { animation-delay: 0ms; }
.stat-card:nth-child(2) { animation-delay: 50ms; }
.stat-card:nth-child(3) { animation-delay: 100ms; }
```

12.2 Responsive design

Les media queries adaptent l'affichage selon la taille d'ecran :

```
/* Desktop par default */
.shell {
  grid-template-columns: 260px 1fr; /* sidebar 260px + contenu */
}

/* Tablette */
@media (max-width: 1024px) {
  .topbar { padding: 14px 20px; }
  .content { padding: 22px; }
}

/* Mobile */
@media (max-width: 768px) {
  .shell { grid-template-columns: 76px 1fr; } /* sidebar mini */
  .nav-label, .theme-label { display: none; }
  .status-pill { display: none; }
}
```

12.3 Quelques chiffres

Metric	Valeur
Bundle initial	~175 KB (gzipped : ~50 KB)
Premier rendu (FCP)	< 1 seconde
Time to interactive	< 1.5 seconde
Chaque chunk lazy	10-30 KB
Animations FPS	60 fps stable

Score Lighthouse	90+ Performance
------------------	-----------------

Chapitre 13 - Build et deploiement

13.1 Commandes essentielles

```
# Installation des dependances (premiere fois)
npm install

# Dev server (avec hot reload)
npm start
# ou : npx ng serve
# -> http://localhost:4200

# Build production
npm run build:prod
# -> dist/chatbot-fsbm/ (HTML + JS + CSS minifies)

# Build dev (plus rapide)
npm run build

# Generer un composant (CLI)
npx ng generate component features/admin-dashboard
# ou raccourci : ng g c features/admin-dashboard
```

13.2 Structure du build production

```
dist/chatbot-fsbm/
  index.html           (1 KB)
  main-XXXXXXXXX.js    (175 KB - bundle initial)
  polyfills-XXXXXXXXX.js (90 KB - support vieux navigateurs)
  styles-XXXXXXXXX.css (13 KB)
  chunk-XXXXXXXXX.js    (chunks pages lazy)
  assets/              (images, logos)
```

13.3 Deploiement

En production, on n'utilise PAS le dev server Angular. On sert les fichiers statiques avec :

- **nginx** (recommande, ultra-rapide)
- **Apache httpd** (plus traditionnel)
- **Vercel / Netlify** (sans serveur, automatique)
- **Cloud Storage + CDN** (S3 + CloudFront, GCP, Azure)

Important : il faut configurer le serveur pour **fallback sur index.html** (SPA mode). Sinon, refresh sur /chat donnera un 404.

```
# Exemple nginx
location / {
    try_files $uri $uri/ /index.html;
}
```

13.4 Conclusion

Tu maitrises maintenant le frontend Angular du projet. Pour aller plus loin :

- PDF 03 - pour comprendre le backend qui repond aux appels HTTP
- PDF 07 - pour la communication frontend/backend en detail
- PDF 09 - pour le workflow complet click -> rendu final